

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Isidora Martina Ogas Pavez

11 de septiembre de 2026

18:38

1. Introducción

En este informe pondremos a prueba dos problemas clásicos de la ciencia de la computación: el ordenamiento de arreglos unidimensionales y la multiplicación de matrices cuadradas. Para la primera parte, implementamos los algoritmos Quick Sort, Merge Sort y Patience Sort, sumando también a `std::sort` (que ya venía listo en la base de la tarea). Por el lado de las matrices, enfrentamos al método tradicional (Naive) contra el algoritmo de Strassen.

El objetivo principal de todo este análisis es llevar la teoría a la vida real: queremos contrastar el comportamiento empírico de estas implementaciones, midiendo cuánto tiempo se demoran y cuánta memoria consumen, con su complejidad teórica asintótica. De esta forma, buscaremos comprobar si los modelos matemáticos realmente se sostienen cuando los llevamos a la práctica.

2. Entorno experimental

2.1. Hardware y Software

Para llevar a cabo los experimentos, se utilizó un equipo provisto de un procesador Intel Core i7-11800H (11.^a generación, a 2.30GHz) y 32 GB de memoria RAM. Aunque el sistema operativo base de la máquina es Windows 11 Home Single Language, todo el desarrollo, la compilación y la ejecución de los programas en C++ se realizaron en un entorno de Linux. Esto se logró a través del Subsistema de Windows para Linux (WSL), corriendo la distribución Ubuntu 24.04.2 LTS y utilizando el compilador g++ en su versión 13.3.0.

2.2. Datasets y Metodología

Respecto a los datos, se trabajó exactamente con los conjuntos generados según las especificaciones del enunciado. Para el problema de ordenamiento, probamos arreglos con tamaños que van desde los 10 hasta los 10 millones de elementos ($n \in \{10, 10^3, 10^5, 10^7\}$). Para evaluar cómo reaccionaban los algoritmos frente a distintos escenarios, estos arreglos se crearon en tres configuraciones (ordenados de forma ascendente, descendente y completamente aleatorios) usando dos rangos de valores: el dominio D1 (números del 0 al 9) y el dominio D7 (del 0 a los 10 millones).

Por el lado de la multiplicación de matrices, evaluamos dimensiones cuadradas en potencias de 2, partiendo desde 16×16 hasta llegar a 1024×1024 ($N \in \{2^4, 2^6, 2^8, 2^{10}\}$). Estas matrices se armaron con tres estructuras distintas (densas, dispersas y diagonales) y bajo dos dominios numéricos: D0 ($\{0, 1\}$) y D10 ($\{0, \dots, 9\}$). Finalmente, para asegurar que los resultados fueran consistentes y no se vieran afectados por procesos secundarios del computador, cada combinación exacta de algoritmo y dataset se ejecutó 3 veces, utilizando estas muestras para generar los resultados finales.

3. Resultados y Análisis

3.1. Análisis de Algoritmos de Ordenamiento

Los cuatro algoritmos de ordenamiento muestran, en general, un crecimiento esperado para su complejidad teórica de $O(n \log n)$. Si miramos los tiempos de MergeSort, QuickSort y `std::sort` crecen de manera practicamente paralela entre los cuatro tamaños evaluados ($n = 10, 10^3, 10^5, 10^7$), ya que los cuatro comparten el mismo orden asintótico. En la práctica, Quick Sort y `std::sort` son los más rápidos. La razón es que ambos ordenan los datos ahí mismo (*in-place*), ahorrándose el trabajo extra de pedir memoria y mover datos hacia estructuras auxiliares, cosa que Merge Sort sí tiene que hacer obligatoriamente.

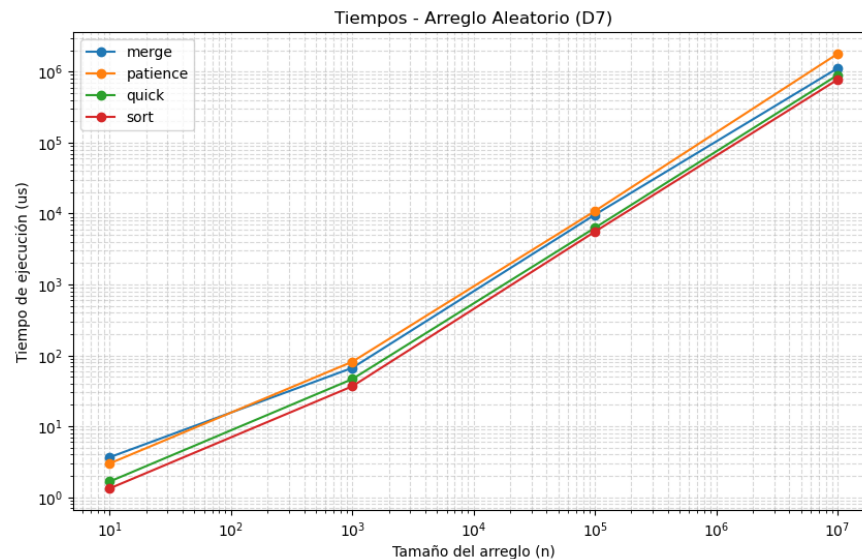


Figura 1: Tiempo de ejecución para arreglos aleatorios (Dominio D7).

El caso de Patience Sort es el más interesante de todos, porque su tiempo de ejecución no depende solo de la cantidad de datos (n), sino de qué tan desordenados vienen. Por ejemplo, en el dominio D7, para $n = 10^7$, tarda cerca de 1.5 segundos (aproximadamente 1543796 us) en el caso ascendente, pero solo del orden de cientos de milisegundos en el caso descendente, una diferencia que no se observa en ningún otro algoritmo. Lo podemos explicar por como va el algoritmo funcionando internamente por que este va armando pilas como si fueran cartas de un solitario, si el arreglo viene de menor a mayor, como cada número es más grande que el anterior tiene que crear una pila completamente nueva por cada elemento (haciendo n pilas en total, su peor caso). En cambio, si viene de mayor a menor, todos los elementos caben perfectamente uno sobre otro en una sola pila. Como la complejidad real de Patience Sort es $O(n \log k)$, con k el número de pilas resultante, y no simplemente $O(n \log n)$, este experimento demuestra de forma muy clara la diferencia entre la teoría general y el comportamiento del algoritmo según cómo vengan los datos de entrada.

En cuanto al uso de memoria, QuickSort es consistentemente el más eficiente, coherente con ser el único que ordena completamente *in-place*. Patience Sort es el que más recursos consume: al mantener muchas pilas separadas usando arreglos bidimensionales (`vector<vector<int>>`), gasta muchísima más memoria y tiempo de asignación que si usara un solo arreglo largo y continuo, además de requerir un *heap* extra. MergeSort también supera a QuickSort por la reserva del arreglo auxiliar de tamaño n que usa para la mezcla.

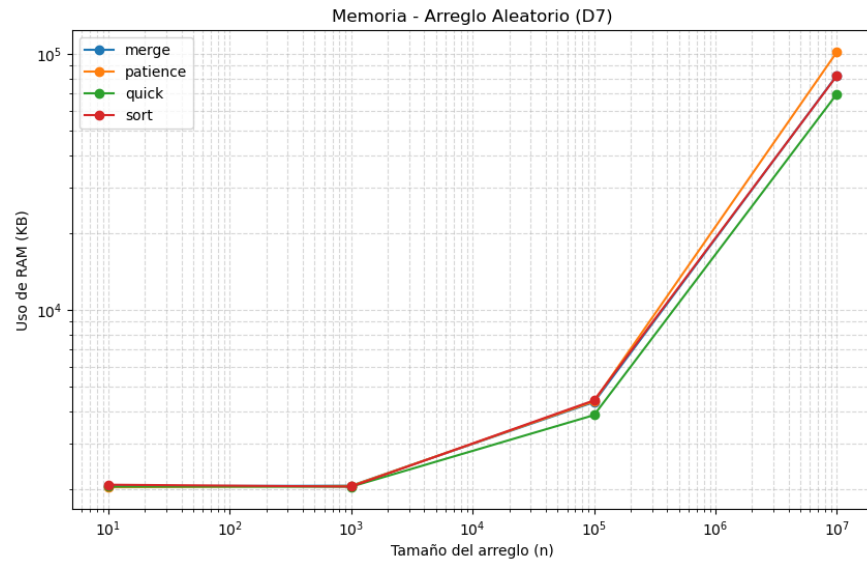


Figura 2: Tiempo de ejecución para arreglos aleatorios (Dominio D7).

Un caso particular es `std::sort`: pese a ser *in-place*, su consumo de memoria es similar al de MergeSort, porque la función `sortArray` recibe el arreglo original, pero luego retorna una copia completa, disparando el uso de memoria de forma artificial.

3.2. Análisis de Multiplicación de Matrices

Se comparó el tiempo de ejecución de Naive y Strassen para matrices cuadradas de dimensión $N \in \{2^4, 2^6, 2^8, 2^{10}\}$. Para calcular la complejidad de Strassen se puede utilizar el **Teorema Maestro** a partir de su recurrencia $T(N) = 7T(N/2) + O(N^2)$: con $a = 7$, $b = 2$, $f(N) = O(N^2)$ y $d = 2$, se compara d con el $\log_b a$; como el $\log_2 7$ es mayor que d , cae en el Caso 1 del teorema, obteniendo un tiempo de ejecución de $T(N) = \Theta(N^{2.81})$. Naive, en cambio, al estar implementado mediante tres ciclos `for` anidados sin recursión, no se resuelve mediante el **Teorema Maestro**, ya que su complejidad $\Theta(N^3)$ se obtiene directamente contando las $\Theta(N^3)$ operaciones de los ciclos.

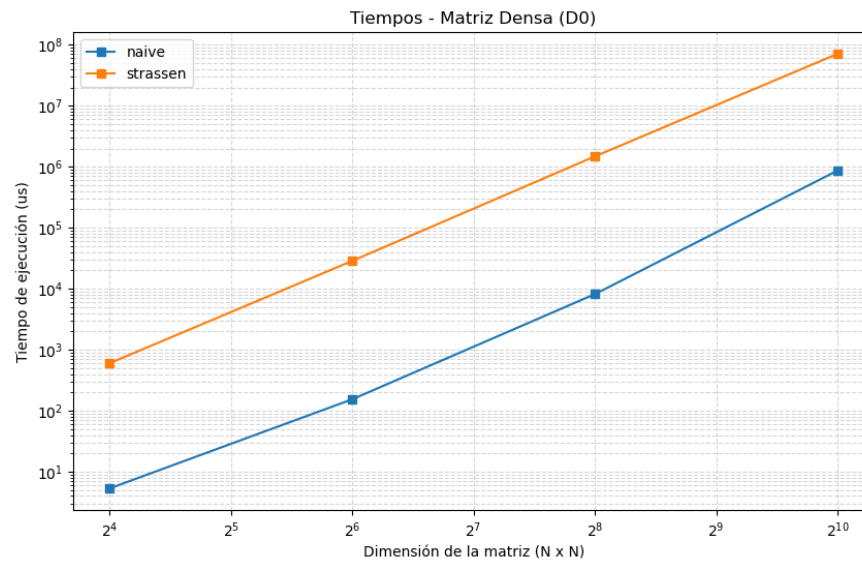


Figura 3: Tiempo de ejecución en matrices densas (Dominio D0).

A pesar de que en el papel Strassen ($N^{2,81}$) debería ganarle a Naive (N^3), en la práctica resultó sistemáticamente más lento en todas nuestras pruebas, sin importar el tipo de matriz ni el dominio. Esto se explica por dos grandes factores:

1. El trabajo extra: aunque Strassen se ahorra 1 multiplicación (hace 7 en vez de 8), realiza cerca de 18 sumas y restas de matrices enteras en cada paso recursivo, lo cual es muy pesado y hace que se demore, en tiempo, más que Naive en casos grandes.

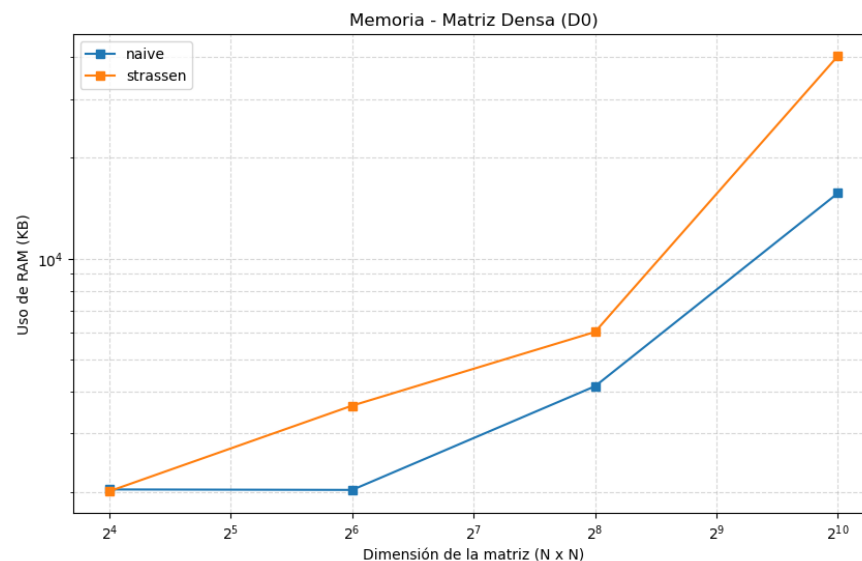


Figura 4: Consumo de memoria en matrices densas (Dominio D0).

2. El problema de cómo está programado: ambos algoritmos representan las matrices como `vector<vector<int>>` (un vector bidimensional). Esta forma de guardar datos hace que las filas queden dispersas en la

memoria, provocando que al procesador le cueste más encontrarlas (mala localidad de caché). El detalle es que Naive reserva su memoria una sola vez al principio, mientras que Strassen, al ser recursivo, crea y destruye decenas de matrices pequeñas temporales en cada uno de sus niveles. Esto multiplica la cantidad de veces que el sistema operativo debe asignar memoria, disparando tanto el tiempo real de ejecución como el consumo de RAM.

En resumen, para que el algoritmo de Strassen empiece a ser más rápido que Naive en la vida real, se necesitan matrices muchísimo más gigantes que las de este laboratorio, además de programarlo usando arreglos planos.

4. Repositorio

A continuación se deja el repositorio. <https://github.com/IsiOgas/T01AlGOCO>

5. Conclusiones

Como vimos en los resultados, la complejidad teórica casi siempre predice bien lo que pasa en la práctica: MergeSort, QuickSort y `std::sort` escalan tal como se espera con su $O(n \log n)$, y lo mismo pasa con Naive y su $O(N^3)$. Sin embargo, hubo dos casos que nos demostraron que quedarse solo con la notación asintótica no basta para explicar el rendimiento real.

Por un lado, Patience Sort (que es $O(n \log k)$ dependiendo de la subsecuencia creciente más larga) tuvo tiempos radicalmente distintos dependiendo de si el arreglo venía ordenado al derecho o al revés. Por otro lado, Strassen nos dio la gran sorpresa: aunque en el papel tiene un exponente asintótico mejor, en la práctica fue sistemáticamente más lento que Naive en todo el rango evaluado por culpa del costo constante de sus sumas adicionales de submatrices. Ambos casos respaldan perfecto la premisa central de esta tarea: ir más allá de la fórmula matemática es una obligación si de verdad queremos entender cómo se va a portar un algoritmo en el computador.